

EXPERIMENT-13 Minimax Algorithm for Gaming

Aim

To implement the **Minimax algorithm** in Python for selecting the optimal move in a two-player game.

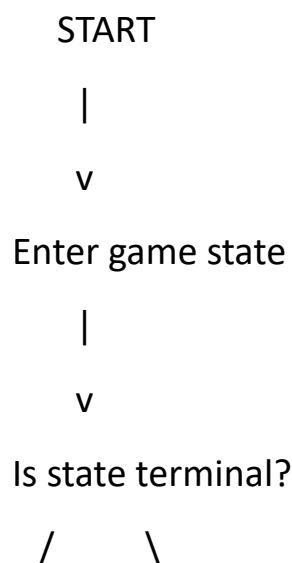
Objectives

- To understand adversarial search.
- To implement MAX and MIN decision-making.
- To select the best possible move assuming the opponent also plays optimally.

Algorithm

1. Start with the current game state.
2. Generate all possible moves.
3. If the state is terminal, return its utility value.
4. For MAX player, select the maximum value.
5. For MIN player, select the minimum value.
6. Continue recursively until the best move is found.
7. Display the optimal move.

Flowchart



Yes No

| |

v v

Return utility Generate

value possible moves

|

v

MAX or MIN?

/ \

MAX MIN

| |

Select maximum Select minimum

\ /

\ /

v v

Return value

|

v

Display best move

|

v

STOP

Python Program

```
def minimax(depth, node_index, maximizing_player, values):  
    # Leaf node  
    if depth == 3:  
        return values[node_index]  
  
    if maximizing_player:  
        return max(  
            minimax(depth + 1, node_index * 2, False, values),  
            minimax(depth + 1, node_index * 2 + 1, False, values)  
        )  
    else:  
        return min(  
            minimax(depth + 1, node_index * 2, True, values),  
            minimax(depth + 1, node_index * 2 + 1, True, values)  
        )
```

```
values = [3, 5, 2, 9, 12, 5, 23, 23]
```

```
best_value = minimax(0, 0, True, values)
```

```
print("Leaf node values:", values)
```

```
print("Best value using Minimax:", best_value)
```

Output

Leaf node values: [3, 5, 2, 9, 12, 5, 23, 23]

Best value using Minimax: 12

Output:

```
main.py
1 def minimax(depth, node_index, maximizing_player, values):
2     # Leaf node
3     if depth == 3:
4         return values[node_index]
5
6     if maximizing_player:
7         return max(
8             minimax(depth + 1, node_index + 2, False, values),
9             minimax(depth + 1, node_index + 2 + 1, False, values)
10        )
11    else:
12        return min(
13            minimax(depth + 1, node_index + 2, True, values),
14            minimax(depth + 1, node_index + 2 + 1, True, values)
15        )
16
17 values = [3, 5, 2, 9, 12, 5, 23, 23]
18 best_value = minimax(0, 0, True, values)
19
20 print("Leaf node values:", values)
21 print("Best value using Minimax:", best_value)
22
23 Press (Ctrl) (C) to generate code
```

```
TERMINAL PROBLEMS
Leaf node values: [3, 5, 2, 9, 12, 5, 23, 23]
Best value using Minimax: 12

** Process exited - Return Code: 0 **
```

Result

The Minimax algorithm successfully determines the **optimal value as 12**.

Conclusion

Thus, the **Minimax algorithm** was successfully implemented to determine the best decision for a player assuming an optimal opponent.